SQL Server 2014

Hekaton: performance estreme con la tecnologia in-memory in modo trasparente ed integrato

Gianluca Hotz

ghotz@ugiss.org

Speaker

▶ Gianluca Hotz

Consultant and mentor at SolidQ
- Administration, capacity planning, data modeling, development, optimization
- 18 years on SQL Server (from 4.21 in 1996)

Interests
- Relational model, DBMS architectures, high availability and disaster recovery

Microsoft MVP SQL Server since 1998

Founder & vice-president UGISS
- User Group Italiano SQL Server
- Official Italian PASS Chapter

Mail: ghotz@ugiss.org - ghotz@solidq.com

Agenda

Introduction

In-Memory Storage

On-Disk Storage (credits: Igor Pagliai)

Native Compilation (credits: Igor Pagliai)

In-Memory OLTP

Code name «Hekaton»

- Originally aimed at 100x

Need to better exploit RAM/CPU

- RAM cost constantly decreasing
- CPU frequency no more growing exponentially
- But number of cores still growing
- Need a technology to execute the same operations in less CPU cycles

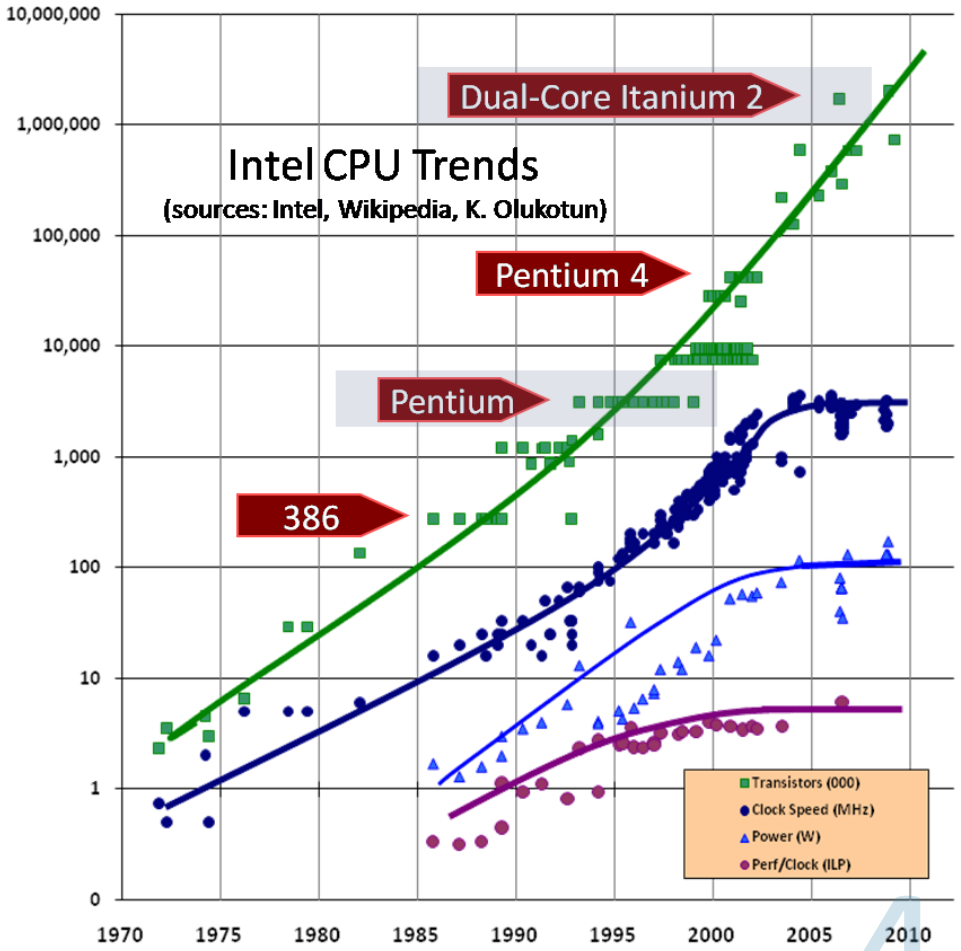


Chart source: http://gotw.ca/publications/concurrency-ddj.htm

Main components

Memory Optimized Tables

Native Compiled Stored Procedures

Lock & latch free data structures

Complete integration

What is not

Something similar to DBCC PINTABLE

A NoSQL solution

Something dangerous because data is volatile

A separate product

A solution for all scenarios

What about ACID compliance?

Atomicity
- Always guaranteed (rollback/rollforward/crash recovery)

Consistency
- Always guaranteed

Isolation
- Always guaranteed (MVCC lock-free consistent access)

Durability*
- You choose: schema only or schema and data

Table storage

Completely different from disk-based tables

- Organized to exploit byte-addressable memory
- Not allocated from Buffer Cache
- At least one index is needed
  - *tables are collections of rows connected by indexes*
- Operations are lock & latch free!

Many ways to block

Locking

- Rows, key ranges, pages, tables, databases
- Shared, exclusive, update, schema modify, intent

Latching

- Light-weight construct for physical structures

Spin locking

- Retry-logic with increasing waiting timers and yielding

Lock-free structures

Mainly due to new MVCC protocol

- Timestamp based
- Multiple row versions
- Garbage collection of obsolete versions

Optimistic

- Delayed validation at commit
- Write conflicts are possible
- Retry logic needed (since a long time anyway…)

Row Format

Row header	Payload (table columns)

Completely new

Basic structure Row Header + Payload

Payload format can vary depending on table

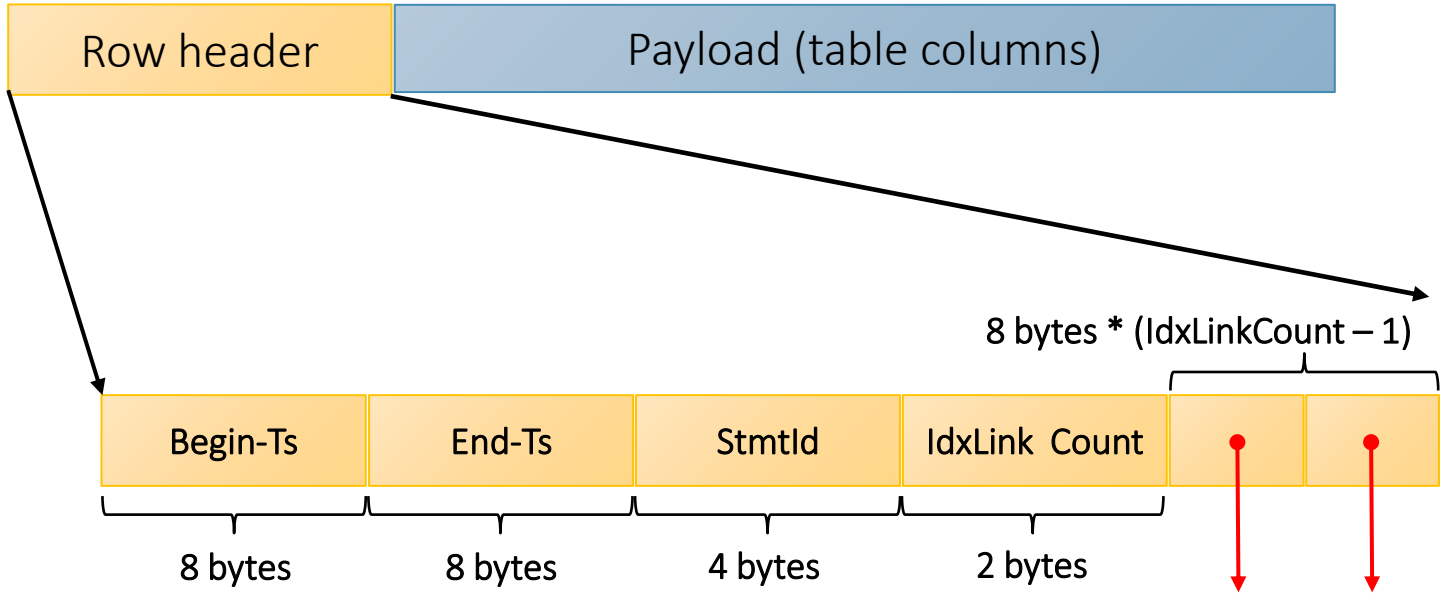
Row Header

Timestamps

Statement

Index count*

Index pointers



Drawing source: http://channel9.msdn.com/Events/TechEd/NorthAmerica/2013/DBI-B308

Row Visibility

Valid Time

- Begin-Ts & End-Ts

Logical Read Time

Visibility

- Begin-Ts < LRT and End-Ts > LRT

Index Types

Hash indexes

- Efficient for equality searches

BW-Tree

- Efficient for range and equality searches

Exists only in memory

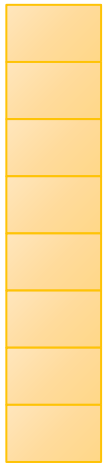
- Only modification to rows written to disk (if durable)
- Recreated during recovery (if durable)

Timestamps	Chain ptrs	Name	City

Hash indexes

Array of hash buckets with physical pointers

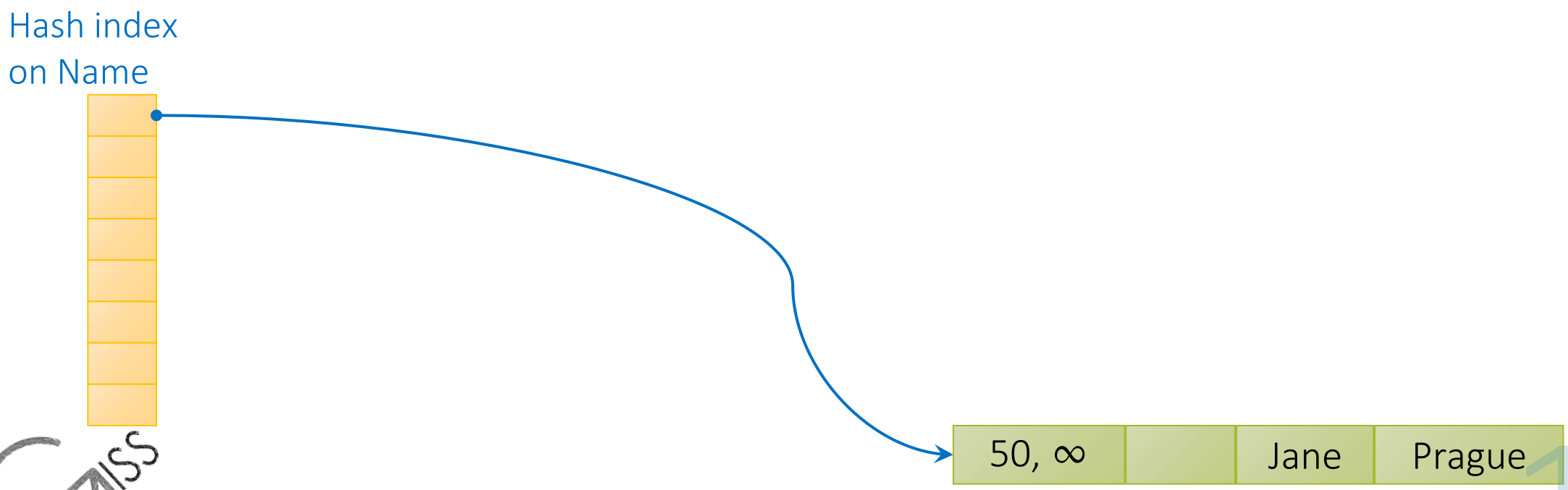
Hash index
on Name



Hash indexes

Timestamps	Chain ptrs	Name	City

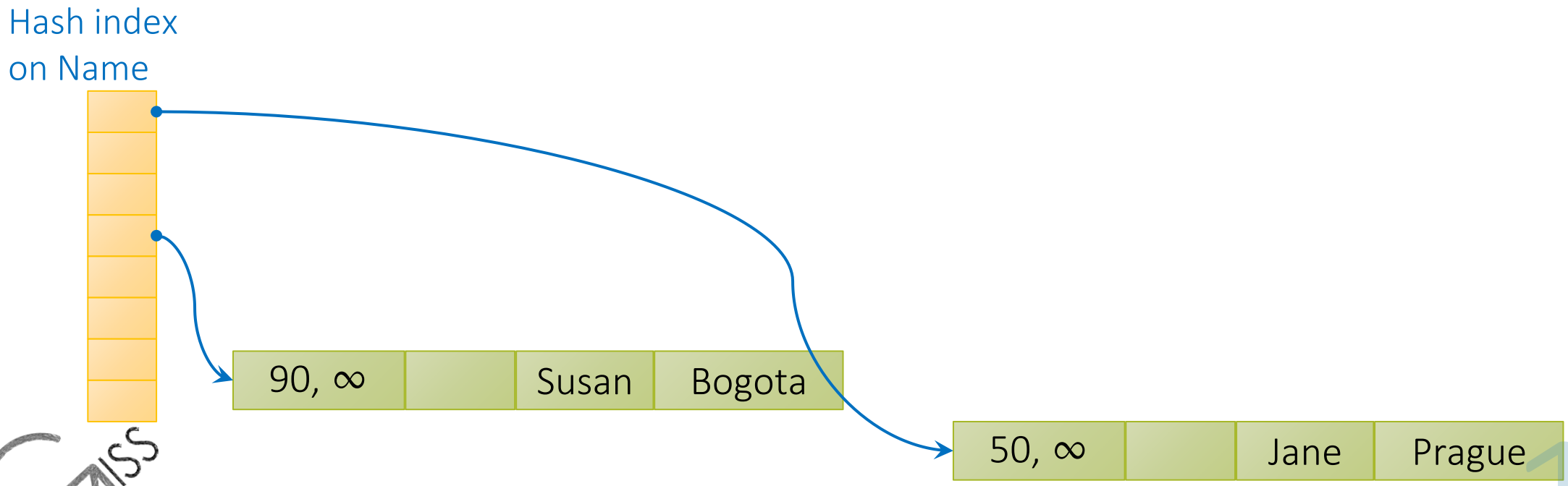
T50: INSERT (Jane, Prague)



Hash indexes

Timestamps	Chain ptrs	Name	City

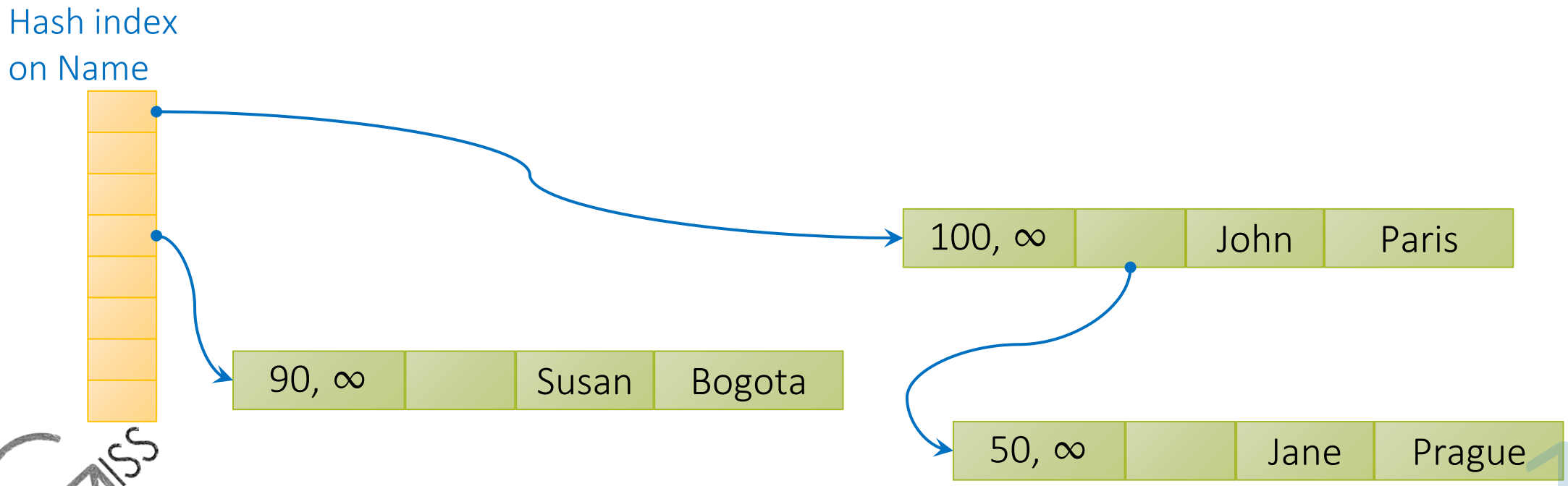
T90: INSERT (Susan, Bogota)



Hash indexes

Timestamps	Chain ptrs	Name	City

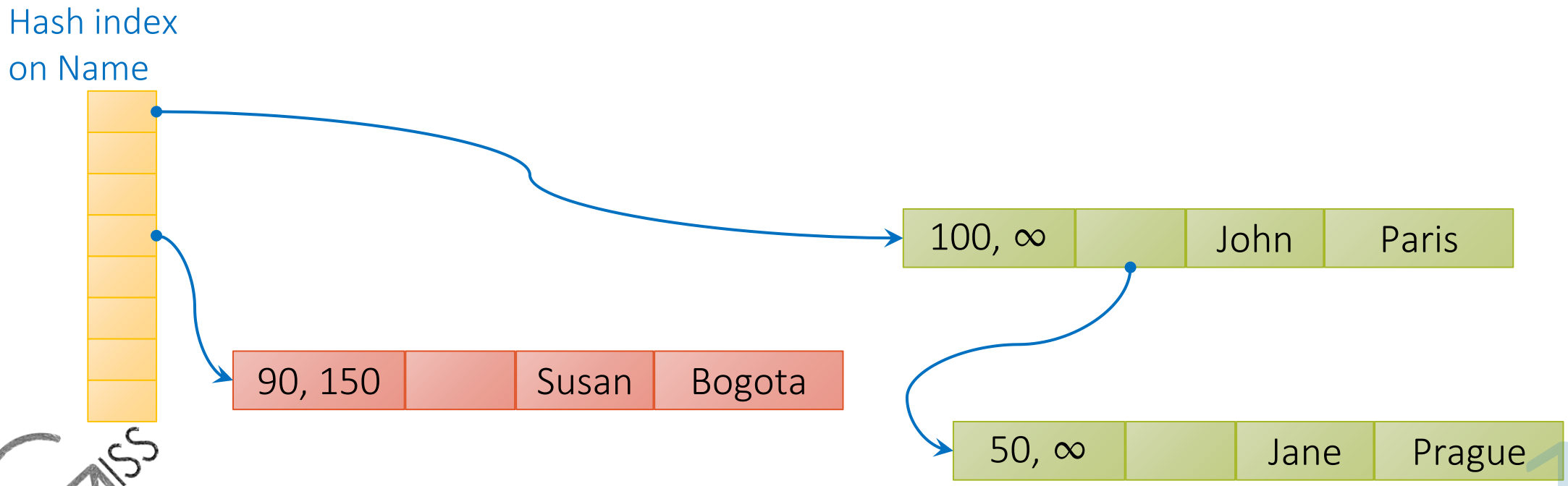
T100: INSERT (John, Paris)



Hash indexes

Timestamps	Chain ptrs	Name	City

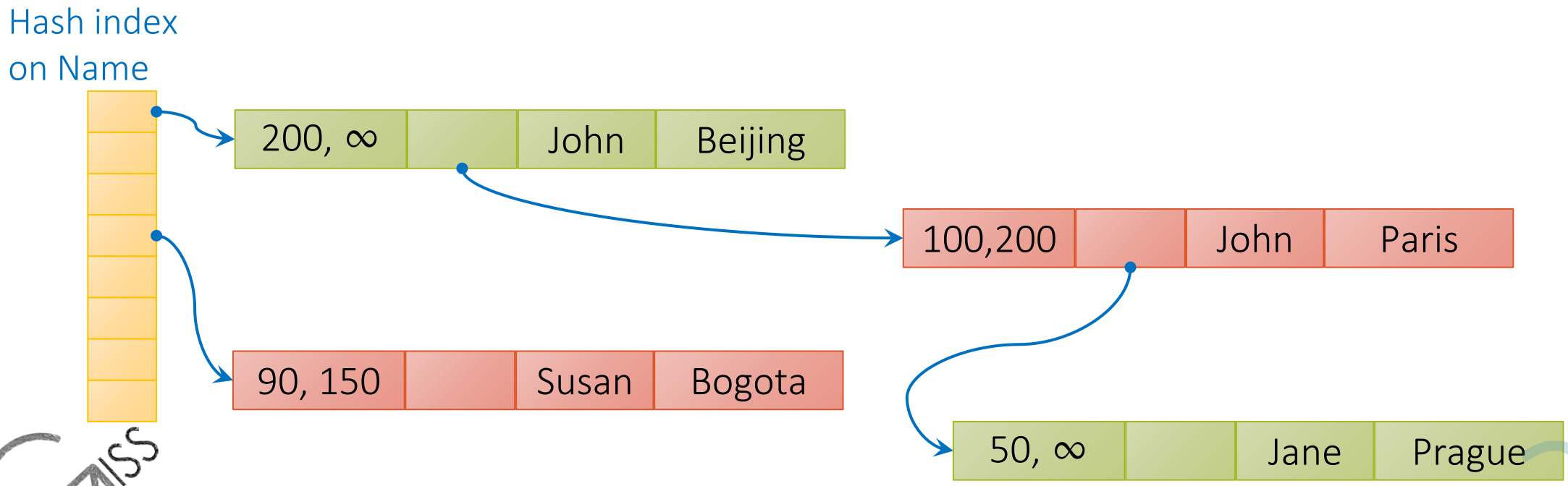
T150: DELETE (Susan, Bogota)



Hash indexes

Timestamps	Chain ptrs	Name	City

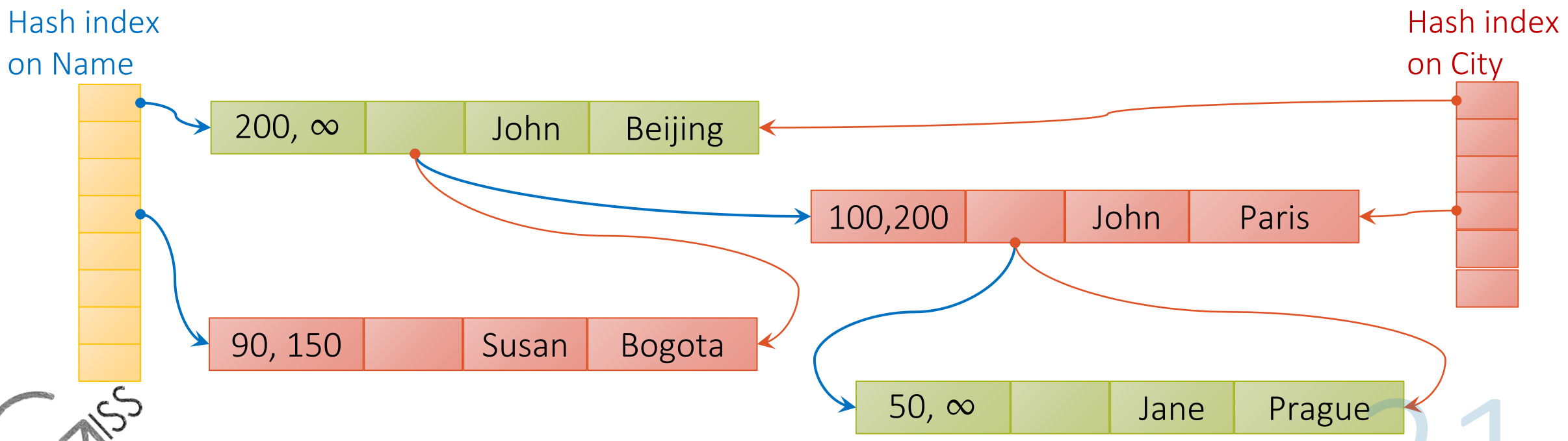
T200: UPDATE (John, Paris) -> (John, Beijing)



Hash indexes

Timestamps	Chain ptrs	Name	City

T200: UPDATE (John, Paris) -> (John, Beijing)



Indexes Caveats

Number of hash buckets is important!

- Memory consumption vs. long rows chains
- Ideally # of distinct values
- Rounded up to next power of 2
- Can be set only at creation time

Garbage Collection

Stale rows slow down scans

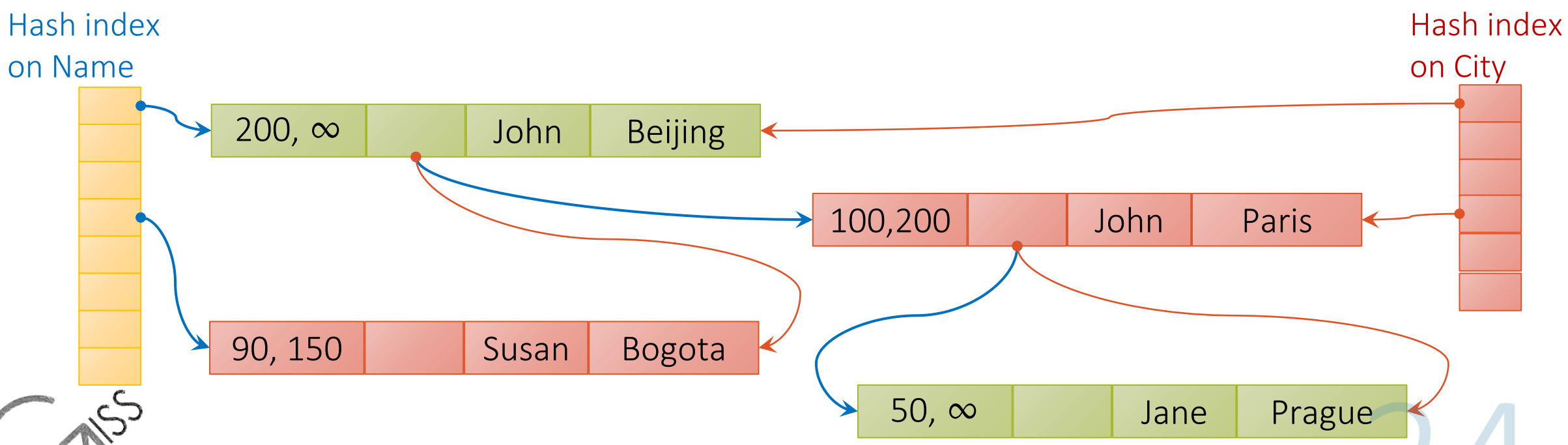
Logical steps

- Determine Oldest Running Transaction
- All rows with End-Ts < ORT-Ts can be removed
- Memory released

GC Example

Timestamps	Chain ptrs	Name	City

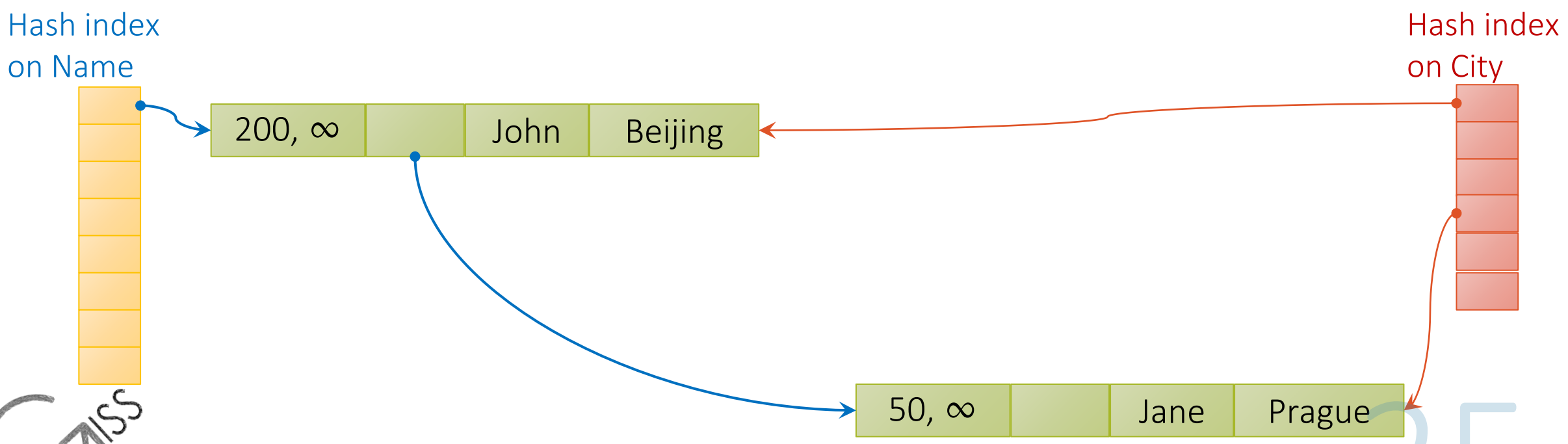
Before Garbage Collection



GC Example

Timestamps	Chain ptrs	Name	City

After Garbage Collection



Cooperative Garbage collection

Scanning user threads do most of the work

- Unlink expired rows and post them to queue
- Free queue work items determined by GC thread

Dedicated system thread

- Runs once per minute (can adjust frequency)
- Builds & distributes work items to be freed to qeues

Wait-free structures

No waiting on latches/spinlocks

Tough problem!

"All problems in computer science can be solved by another level of indirection“ ☺

Wait-free algorithms

Compare-and-swap (CAS) instructions

Longer more complex code

Thread interactions

- Running own code
- Assist in removing other thread blocking condition
- Abort running code (retry if possible)

Wait-free algorithms

Try to think at threading this way

- if thread T1 is doing something and gets pre-empted
- and thread T2 would wait because of T1 activities
- T2 can
  - *assist T1 running code to remove obstructing condition*
  - *abort current activity (eventually retry)*

Range Indexes

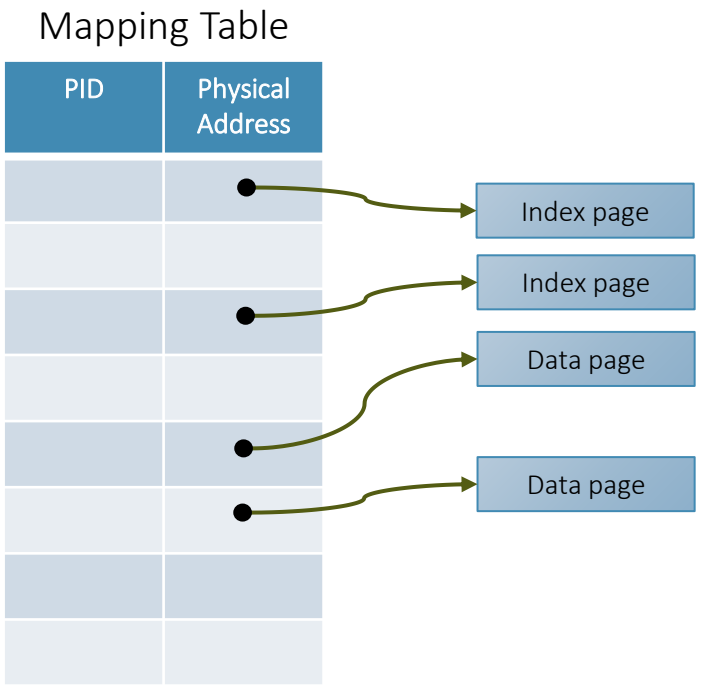
"Buzz Word" Trees ☺ builds on top of B+-Trees

- Introduced in CTP2 as Range Indexes
- Pages are never updated in-place
- Updates generate delta records
- Logical to physical mapping table
- CAS operations

BW-Tree

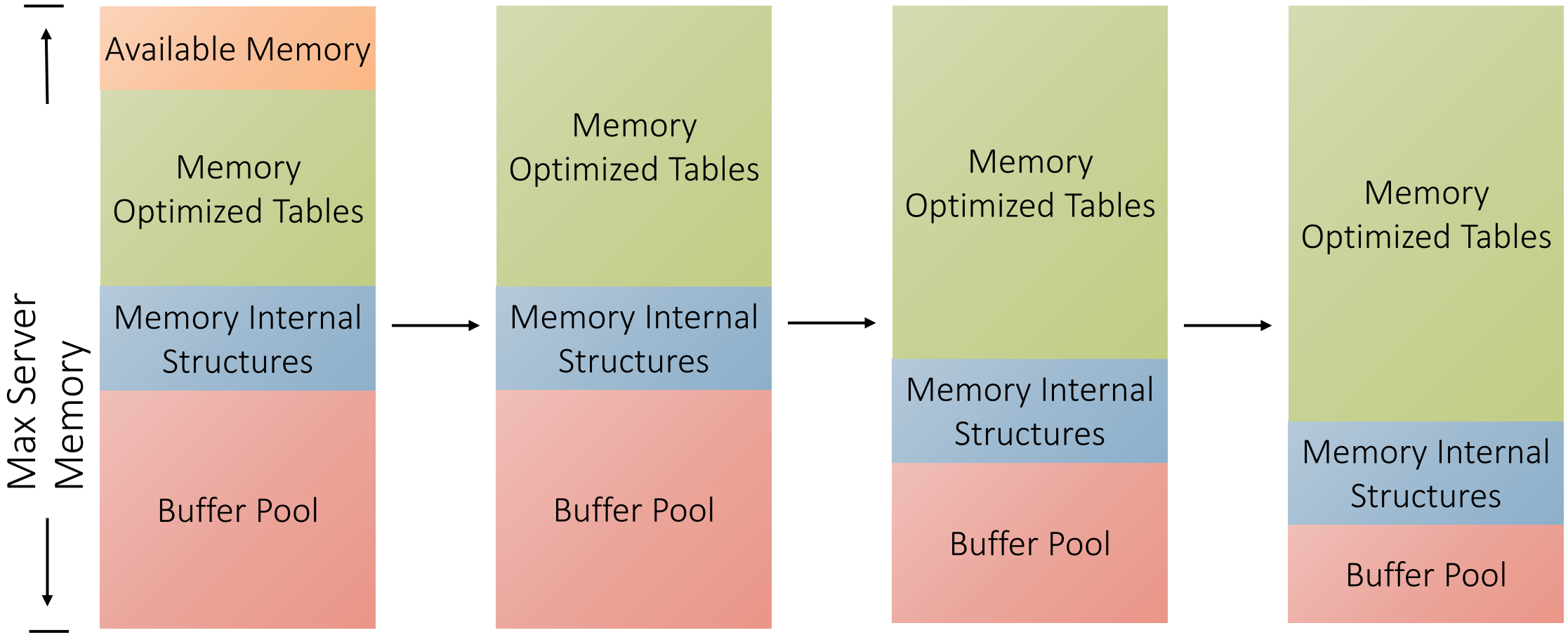
Mapping table

- Logical page ID to physical address
- Key to latch-free behavior
- Isolates update to a single page



Drawing source: http://research.microsoft.com/en-us/um/people/justinle/papers/ICDE2013_bwtree.pptx

Memory challenge



Memory problems

Not enough memory to fit new rows

- Transactions start to fail

Tables don't fit into memory during recovery

- Database offline

Memory pressure caused by Hekaton

- Operations in other workloads may start to fail

Memory problems mitigation

Capacity planning (256GB hard limit per DB)

Monitor

Resource Governor

- Bind database to Resource Pool
- EXEC sp_xtp_bind_db_resource_pool 'dbname', ‘poolname‘
- Need to take database offline and online again

Where DATA resides? Why?

WHAT I'M MISSING?

0 ..... 100 **Transaction Time Stamp Range**

Checkpoint File Pair

Data File

TS (ins)	RowId	TableId	Row pay load
TS (ins)	RowId	TableId	Row pay load
TS (ins)	RowId	TableId	Row pay load

Data file contains rows *inserted* within a given transaction range

Delta File

TS (ins)	RowId	TS (del)
TS (ins)	RowId	TS (del)
TS (ins)	RowId	TS (del)

Delta file contains *deleted* rows within a given transaction range

- ✓ **Offline Checkpoint**
  - 512MB transaction log generation
- ✓ **Pre-allocated pairs**
  - 8 file pairs minimum!
  - RAM <= 16GB
    - DATA = 16MB, DELTA = 1MB
  - RAM > 16GB
    - DATA = 128MB, DELTA = 8MB
- ✓ **I/O pattern**
  - Optimized for sequential write
  - 256KB I/O size, no page structure!
  - Existing data never updated, only appended
- ✓ **Delta files loaded first**
  - Used to filter out data file rows
- ✓ **Updated rows**
  - 1 insert + 1 delete, 0 update!

Transaction Log

No separate storage for Hekaton transaction log

- The real bottleneck: spend your money here for ultra-fast SSD!

Efficient usage by Hekaton

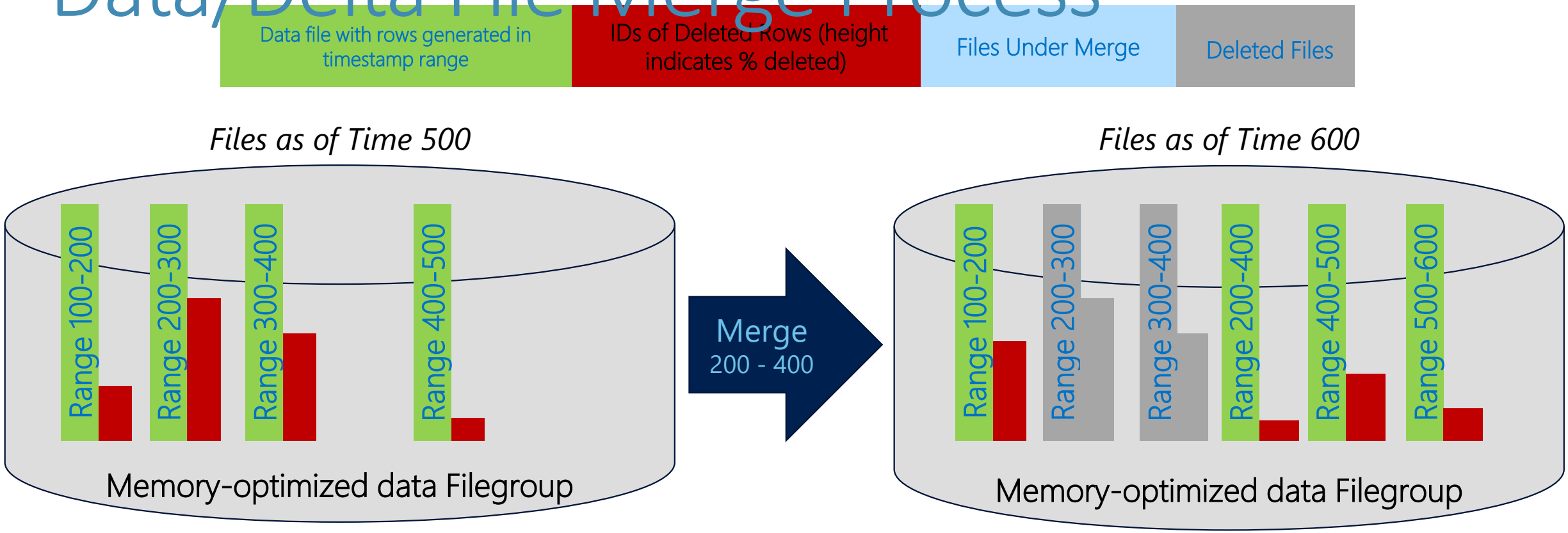
- No Index logging
- No logging on aborted transactions
- No logging on non-durable tables (schema only)
- Up to 20KB-size single log records

Delayed transaction durability

- Faster response time vs. data loss

Group Commit

Data/Delta File Merge Process

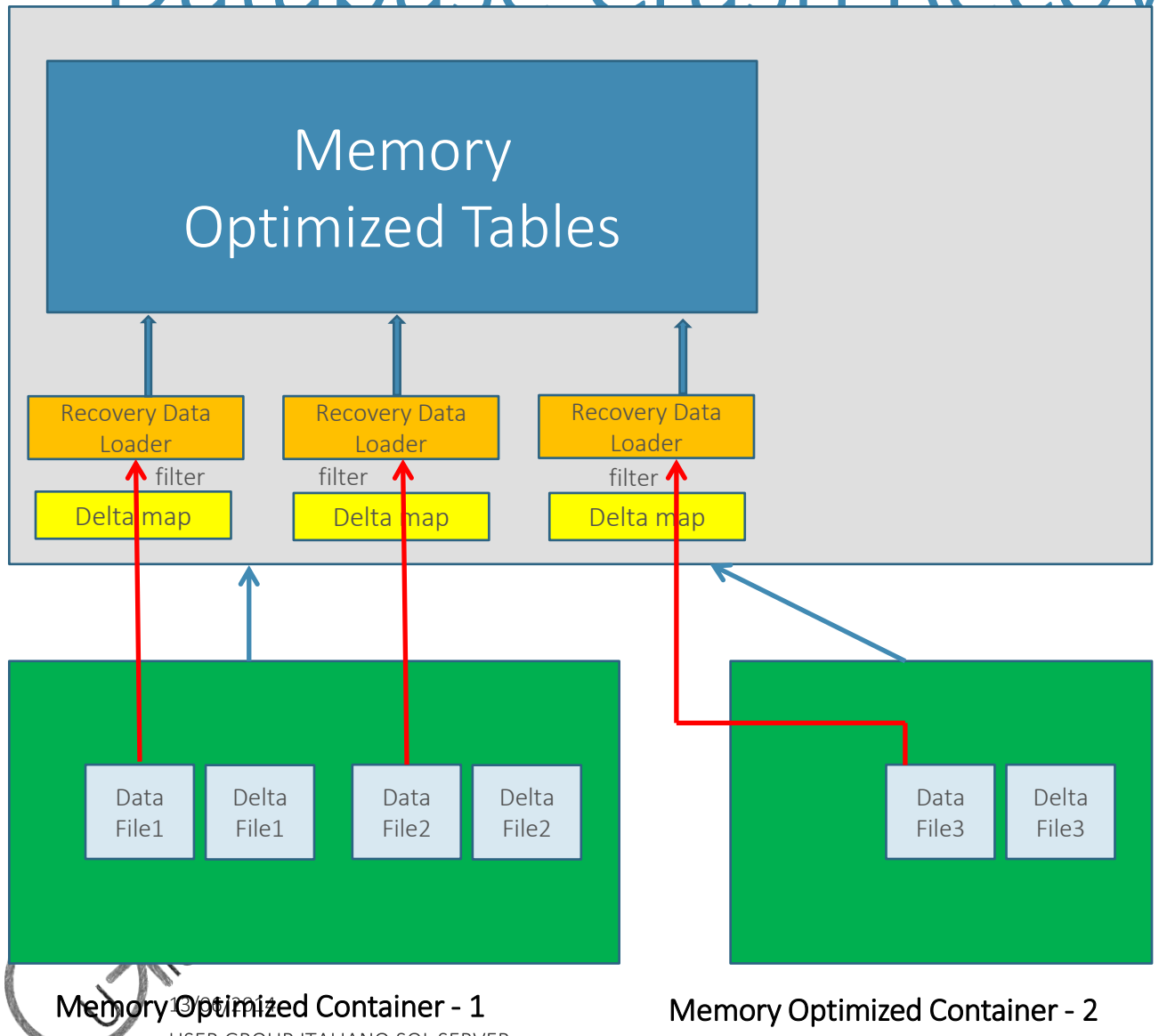


- **Max 4096 pairs → Max 256GB supported (today…) per single DB!**
- **Manual Merge process**
- **List of files maintained in the Checkpoint log record in transaction log**

Backup & Disaster Recovery

- Integrated with SQL Database Backup
  - Memory-Optimized file group is backed up as part SQL database backup
    - CHECKSUM performed and checked on Data/Delta files
    - Transaction log backup includes memory-optimized log records transparently
    - [Allocated Bytes (CTP2) vs. Used Bytes (RTM)] + [File Metadata]
  - Differential backup supported: only new data files since last full backup
    - ALL delta files always included! ***Why?***
    - No page structure → No Differential Maps!
- Database Repair
  - WARNING: No DBCC CHECKDB on data/delta files! You really need it?
  - Don't stop SQL Server! Your data is still valid in memory
  - Failed DB restart/recovery: set Hekaton filegroup offline, but be careful…

Database Crash Recovery



- Parallel data loading & rebuild
  - 1 thread per container to ***read*** data
  - 1 thread per core to ***build*** Hekaton index structures
- Disk performance
  - Optimized for sequential read
- Delta files loaded first
  - Used to filter out data file rows
- Index rebuild processing
  - Created/processed at each restart
  - Many/Bad indexes will slow down database recovery
- Failover Clustering vs. AlwaysOn
  - Which is faster?
- Audit: trace flags & xEvents

```sql
drop table dbo.STDEntities
drop table dbo.HKTEntities
go

-- Create Standard Table
create table dbo.STDentities
(
    customerid int identity not null,
    attribute1 varchar(100) not null,
    attribute2 varchar(100) not null,
    attribute3 varchar(100) not null,
    constraint pk__STDentities primary key nonclustered(customerid)
)
go

-- Create Hekaton Table
create table dbo.HKTEntities
(
    customerid uniqueidentifier not null default(newid()),
    attribute1 varchar(100) not null,
    attribute2 varchar(100) not null,
    attribute3 varchar(100) not null,
    constraint pk__HKTEntities primary key nonclustered hash (customerid)
        with (bucket_count = 50000)
)
with (memory_optimized = on, durability = schema_only)
go

-- Test simple insert
insert into dbo.STDentities (attribute1, attribute2, attribute3) values ('Some', 'Value', 'Here')
insert into dbo.HKTEntities (attribute1, attribute2, attribute3) values ('Some', 'Value', 'Here')
```

Demo

Why Native Compilation?

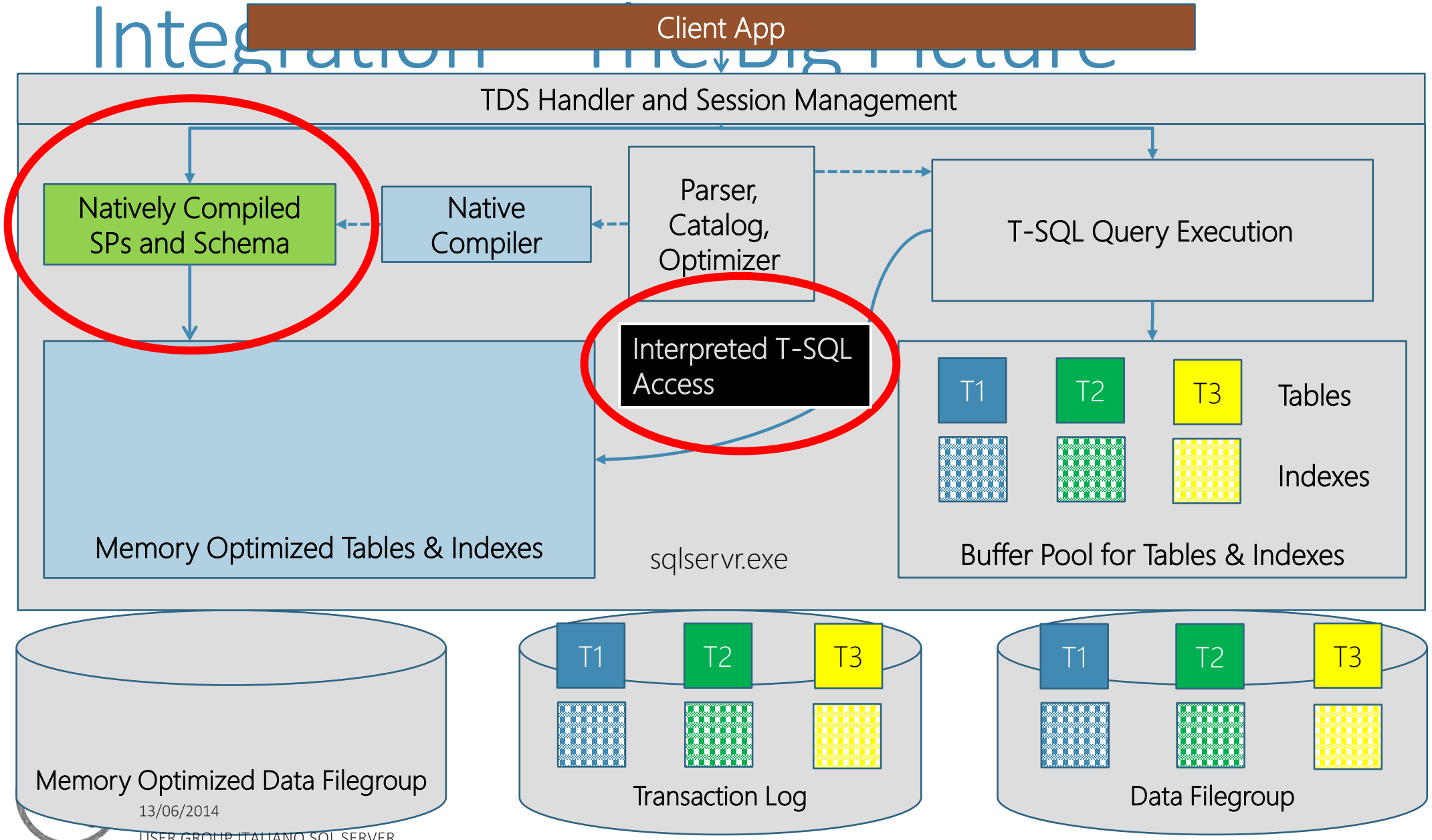
How SQL Server spends its time ?

Move overhead from execution to creation time!

Different kind of compiled objects

- Stored Procedures
- Tables
- …and Table Variables!

Integration – The Big Picture



Existing SQL Component

In-mem OLTP Component

Generated .dll

```sql
drop table dbo.STDEntities
drop table dbo.HKTEntities
go

-- Create Standard Table
create table dbo.STDentities
(
    customerid int identity not null,
    attribute1 varchar(100) not null,
    attribute2 varchar(100) not null,
    attribute3 varchar(100) not null,
    constraint pk__STDentities primary key nonclustered(customerid)
)
go

-- Create Hekaton Table
create table dbo.HKTEntities
(
    customerid uniqueidentifier not null default(newid()),
    attribute1 varchar(100) not null,
    attribute2 varchar(100) not null,
    attribute3 varchar(100) not null,
    constraint pk__HKTEntities primary key nonclustered hash (customerid)
        with (bucket_count = 50000)
)
with (memory_optimized = on, durability = schema_only)
go

-- Test simple insert
insert into dbo.STDentities (attribute1, attribute2, attribute3) values ('Some', 'Value', 'Here')
insert into dbo.HKTEntities (attribute1, attribute2, attribute3) values ('Some', 'Value', 'Here')
```

Demo

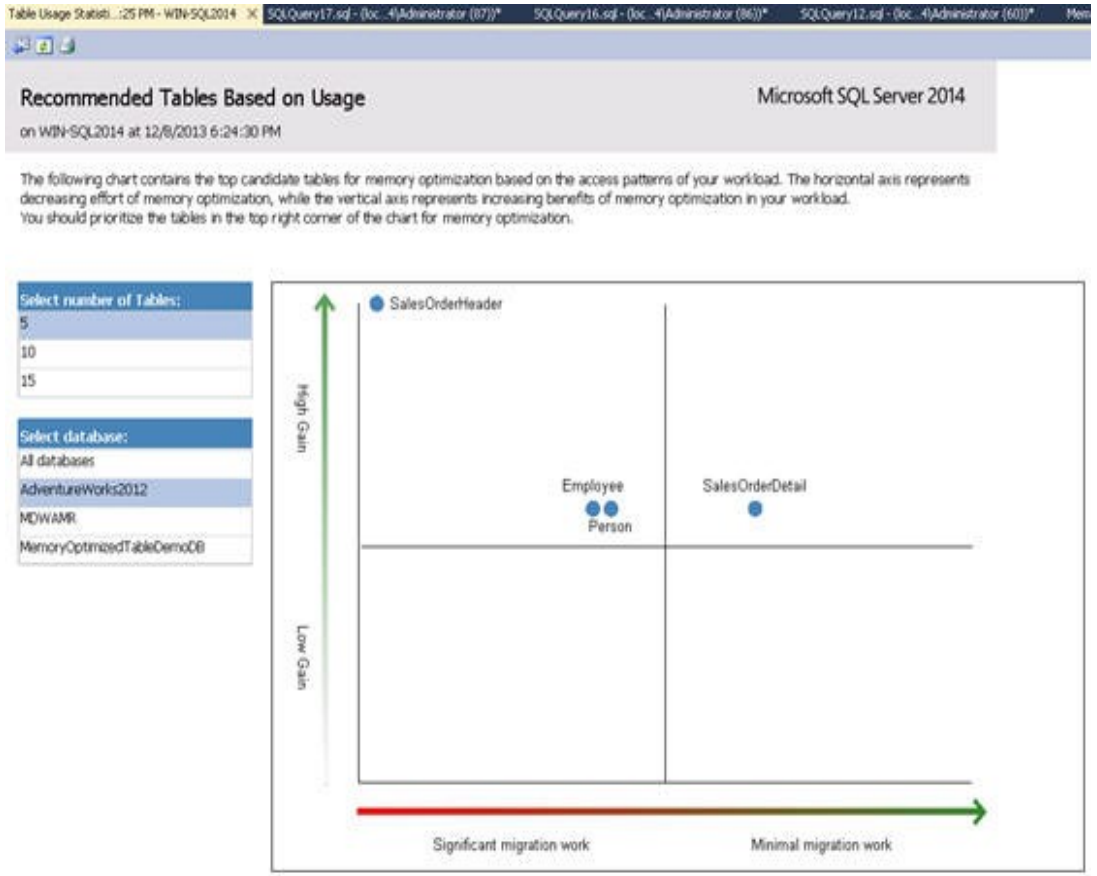
AMR – Analysis Migration & Reporting

Based on Management DWH

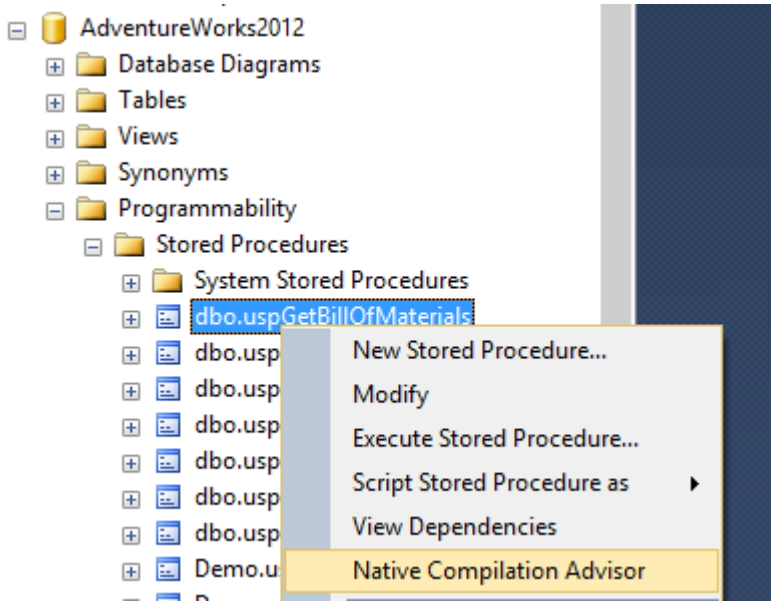
Usage based analysis of

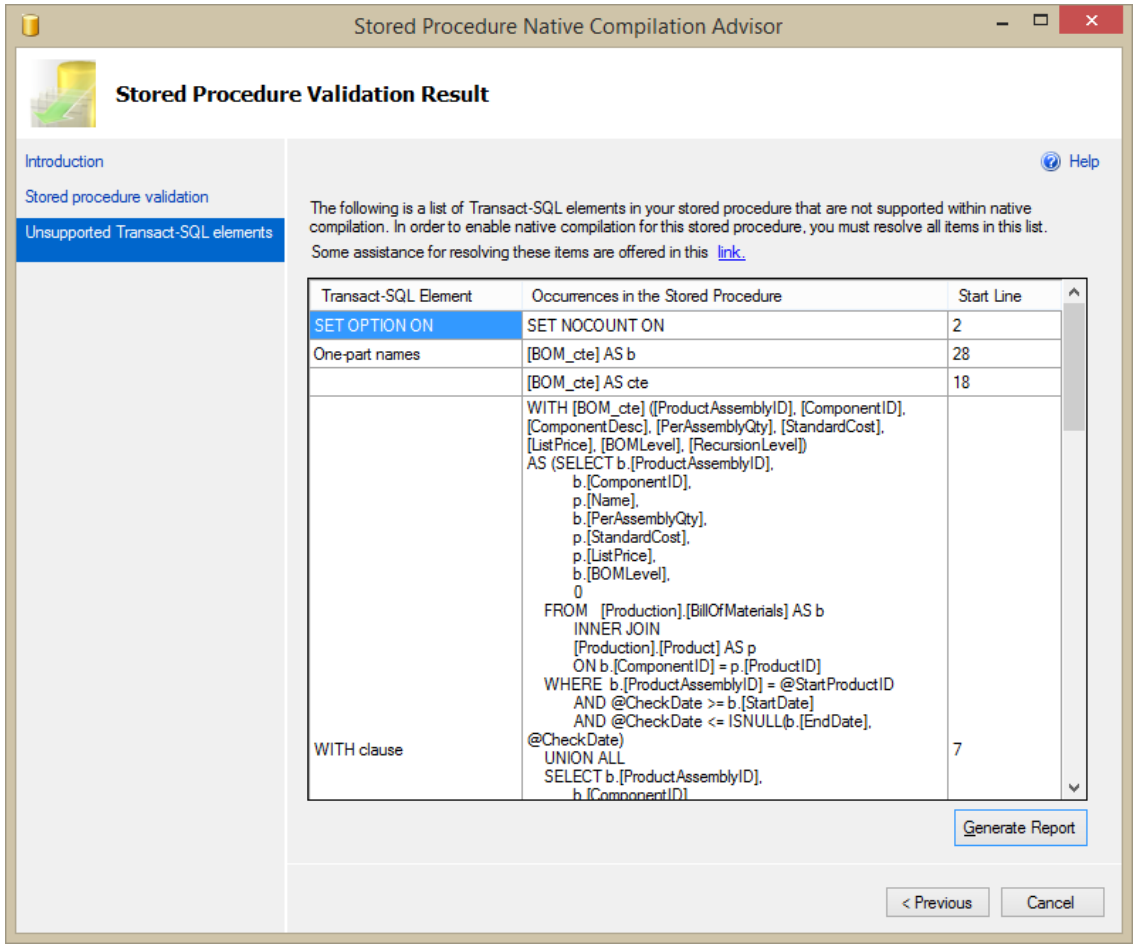
- Tables
- Stored Procedures

Suggest which to migrate



Native Compilation Advisor





Resources

SQL Server In-Memory OLTP Internals Overview for CTP2

Hekaton: SQL Server's Memory-Optimized OLTP Engine

BW-Trees

- The BW-Tree: A B-tree for New Hardware Platforms
  - *Whitepaper*
  - *PowerPoint Presentation*
- The BW-Tree: A Latch-Free B-Tree for Log-Structured Flash Storage
- The BW-Tree: A B-Tree On Steroids

Tony Rogerson Ramblings on Data

- In-Memory tables series

Grazie!

Trovi altri video su:
www.ugiss.org/sql-server-2014-pills